

# Теория параллелизма

## Отчет

### Решение уравнения теплопроводности

Выполнила:  
Южанинова Мария  
Александровна

Группа: 24942

25.05.2026

## Цель работы:

Цель работы — разработать программу для численного решения двумерного уравнения теплопроводности на равномерной сетке методом Якоби с применением пятиточечной схемы, выполнить параллелизацию вычислений средствами OpenACC и провести сравнение производительности CPU и GPU при разных размерах сетки.

## Используемый компилятор:

Для компиляции и запуска программы использовался пакет NVIDIA HPC SDK с компилятором **pgc++**, поддерживающим технологию **OpenACC**. При сборке применялись параметры **-acc** и **-Minfo=all**: первый отвечает за подключение средств OpenACC, второй позволяет получить подробную информацию о процессе распараллеливания и оптимизации кода.

Тестирование программы проводилось в нескольких режимах:

- **-acc=host** — выполнение вычислений без параллелизации на одном ядре CPU;
- **-acc=multicore** — параллельное выполнение на многоядерном центральном процессоре;
- **-acc=gpu** — перенос вычислений на графический ускоритель GPU.

## Используемый профилировщик:

Для исследования производительности приложения применялся профилировщик NVIDIA Nsight Systems. С его помощью проводился анализ времени выполнения программы, степени загрузки графического процессора, а также обмена данными между CPU и GPU.

Профилирование выполняется командой:

```
nsys profile ./app
```

## Как производили замер времени работы:

Измерение времени выполнения программы осуществлялось с использованием средств библиотеки C++ Standard Library **std::chrono**. Фиксация времени производилась до начала вычислительного процесса и после завершения всех итераций алгоритма.

Экспериментальные запуски были выполнены для сеток размером **128×128, 256×256, 512×512 и 1024×1024**. Для каждого размера проводилось сравнение трёх режимов выполнения: последовательного режима CPU, многопоточного режима multicore и вычислений на GPU.

## Выполнение на CPU

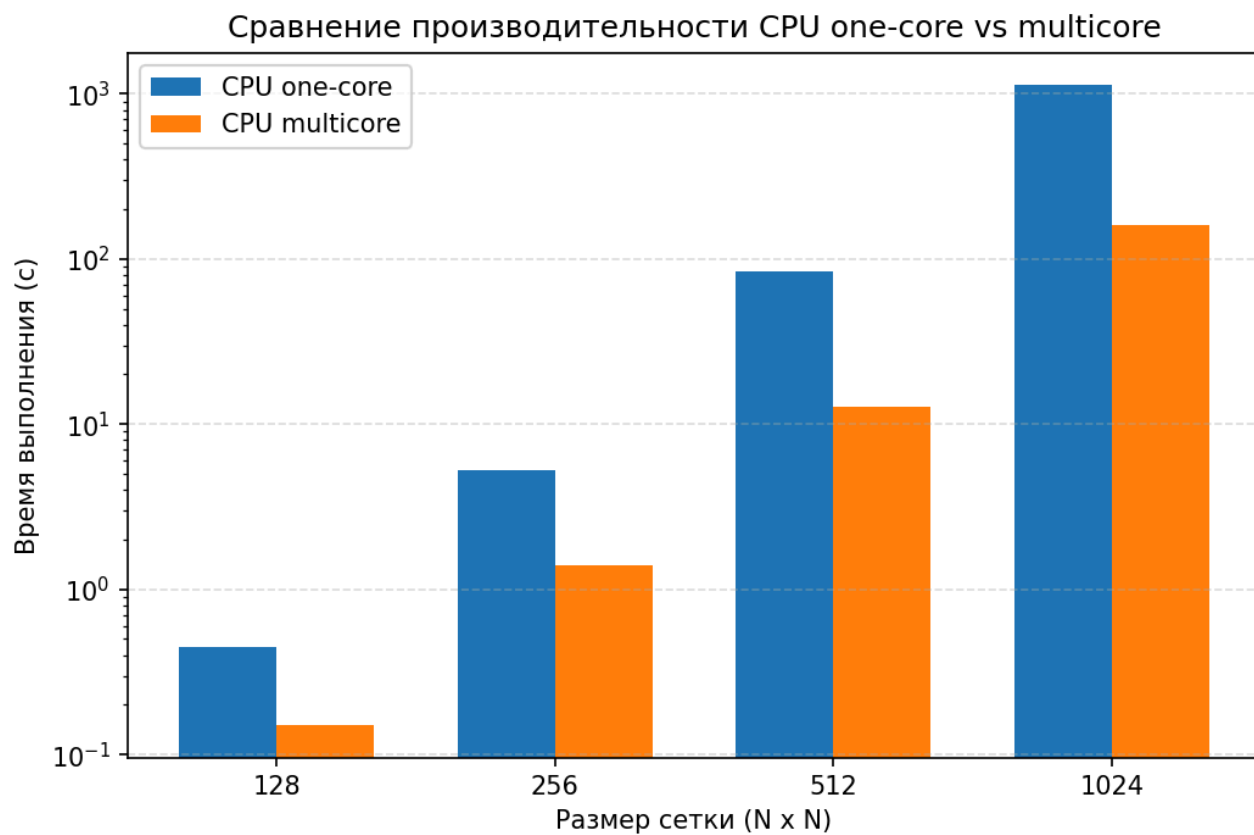
### CPU-onecore

| Размер сетки     | Время выполнения (секунды) | Точность | Количество итераций |
|------------------|----------------------------|----------|---------------------|
| <b>128x128</b>   | 0.4478                     | 0.000001 | 30074               |
| <b>256x256</b>   | 5.2903                     | 0.000001 | 102885              |
| <b>512x512</b>   | 84.4177                    | 0.000001 | 339599              |
| <b>1024x1024</b> | 1129.1313                  | 0.000001 | 1000000             |

### CPU-multicore (8)

| Размер сетки     | Время выполнения (секунды) | Точность | Количество итераций |
|------------------|----------------------------|----------|---------------------|
| <b>128x128</b>   | 0.1501                     | 0.000001 | 30074               |
| <b>256x256</b>   | 1.4054                     | 0.000001 | 102885              |
| <b>512x512</b>   | 12.7194                    | 0.000001 | 339599              |
| <b>1024x1024</b> | 160.6561                   | 0.000001 | 1000000             |

## Диаграмма сравнения время работы CPU-one и CPU-multi

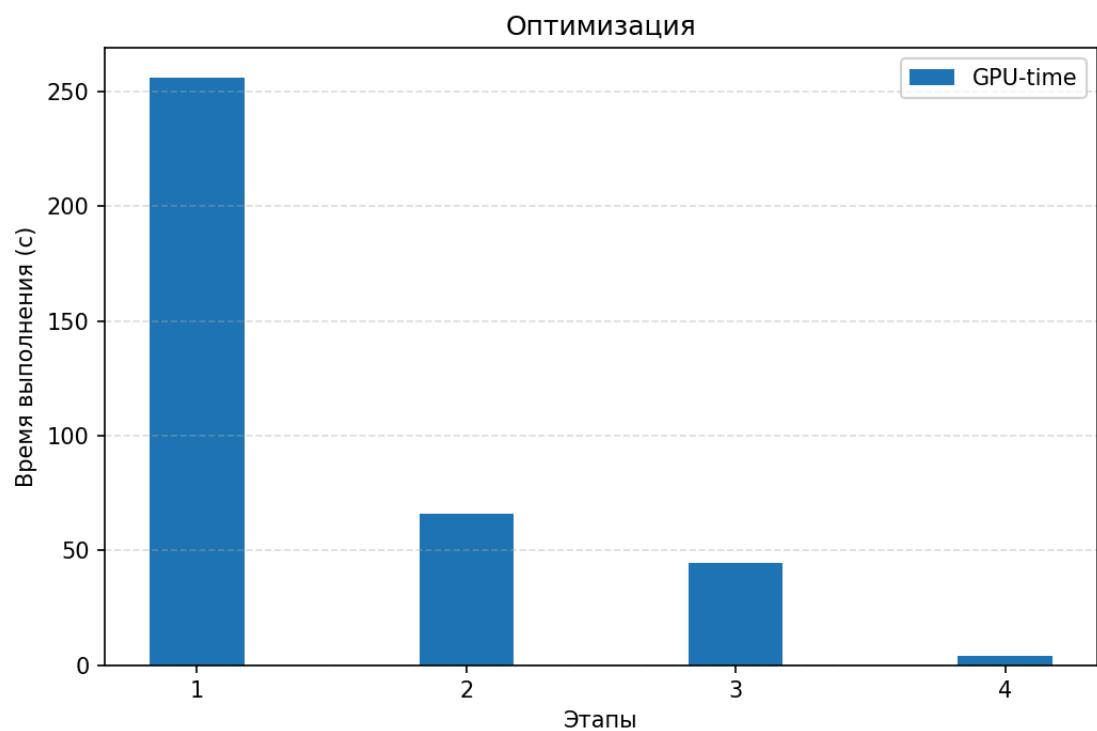


# Выполнение на GPU

## Этапы оптимизации на сетке 512x512

| Этап № | Время выполнения | Точность | Максимальное количество итераций | Комментарий (что было сделано)                                           |
|--------|------------------|----------|----------------------------------|--------------------------------------------------------------------------|
| 1      | 256.3452         | 0.000001 | 1000000                          | Базовая версия                                                           |
| 2      | 65.9928          | 0.000001 | 1000000                          | Убрано лишнее копирование                                                |
| 3      | 44.4765          | 0.000001 | 1000000                          | Оптимизирован доступ к памяти в ядре и упрощены вычисления               |
| 4      | 3.8979           | 0.000001 | 1000000                          | Улучшено распараллеливание циклов и снижены затраты на вычисление ошибки |

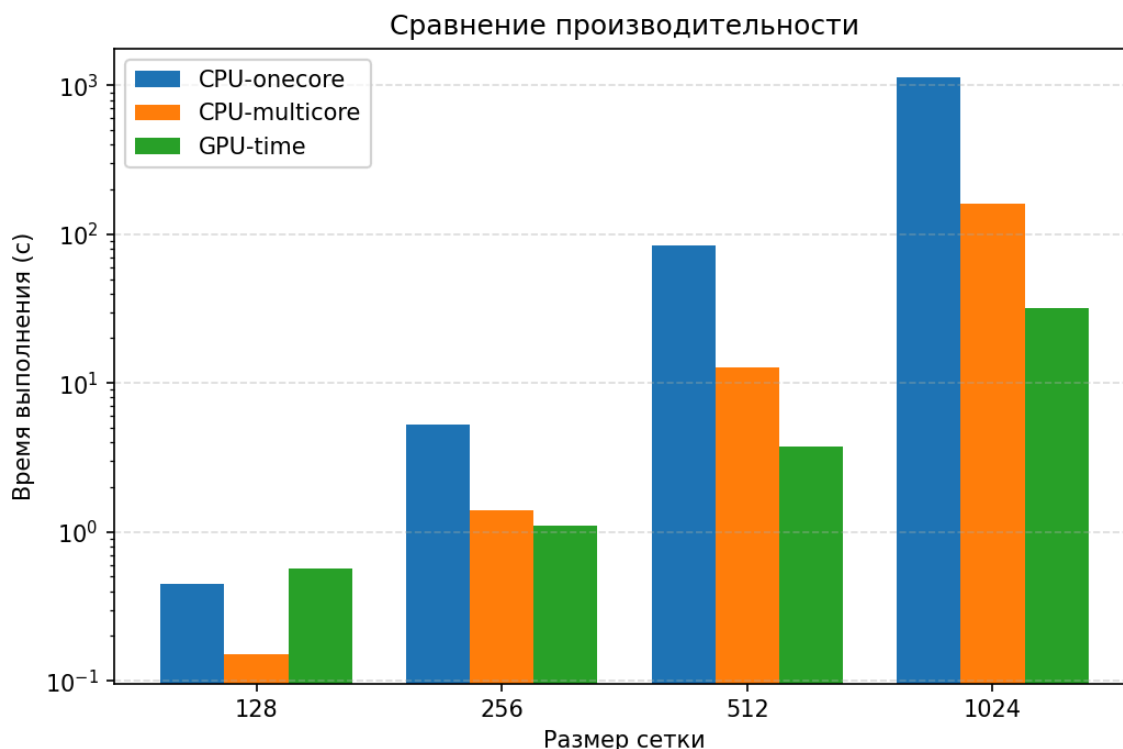
## Диаграмма оптимизации



## GPU – оптимизированный вариант

| Размер сетки | Время выполнения (секунды) | Точность | Количество итераций |
|--------------|----------------------------|----------|---------------------|
| 128x128      | 0.5673                     | 0.000001 | 30101               |
| 256x256      | 1.0989                     | 0.000001 | 102901              |
| 512x512      | 3.7456                     | 0.000001 | 339601              |
| 1024x1024    | 31.6775                    | 0.000001 | 1000000             |

## Диаграмма сравнения времени работы CPU-one, CPU-multi, GPU(оптимизированный вариант) для разных размеров сеток



### Вывод:

По результатам эксперимента наблюдается существенная разница в производительности между однопоточной, многопоточной CPU-реализацией и GPU-вычислениями. Однопоточная версия демонстрирует резкий рост времени выполнения при увеличении размера сетки, что делает ее неэффективной для больших задач. Многопоточная реализация значительно ускоряет вычисления и лучше масштабируется, однако также показывает рост времени при увеличении размерности. GPU-версия обеспечивает наилучшую производительность на больших размерах сетки, демонстрируя лучшую масштабируемость по сравнению с CPU. При малых размерах сетки GPU проигрывает из-за накладных расходов на инициализацию и передачу данных, однако с увеличением задачи становится наиболее эффективным решением.